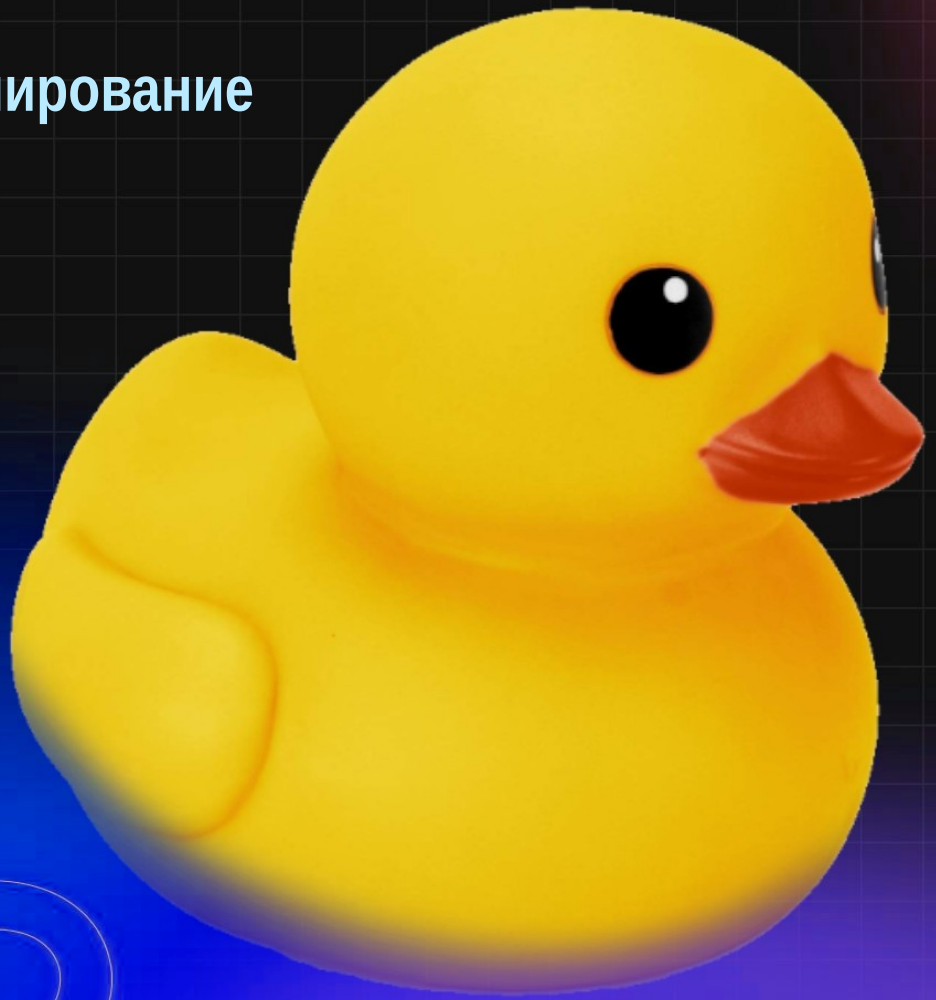
Программирование

2 семестр

2026



Функциональные интерфейсы в Java API

Функциональные интерфейсы

ИТМО

- java.lang.Runnable

  void run();

  new Thread(() -> { … }).start();

- java.util.Comparator<T>

  int compare(T o1, T o2);

  Collections.sort(list, (x,y) -> y - 2 * x);

- java.util.function.* - набор функциональных интерфейсов общего назначения для разных случаев

Java

Пакет java.util.function

- `Supplier<R> { R get() }`
- `Consumer<T> { void accept(T t) }`
- `Predicate<T> { boolean test(T t) }`
- `Function<T,R> { R apply(T t) }`
  - `UnaryOperator<T> { T apply(T t) }`
- `BiFunction<T,U,R> { R apply(T t, U u) }`
  - `BinaryOperator<T> { T apply(T t1, T t2) }`

Supplier

```
Supplier<R> {
  R get()
}

void Logger.log(Level level, Supplier<String> msgSupplier)
```

- `IntSupplier { int getAsInt() }`
- `LongSupplier { long getAsLong() }`
- `DoubleSupplier { double getAsDouble() }`
- `BooleanSupplier { boolean getAsBoolean() }`

Consumer

```
Consumer<T> {
  void accept(T t)
  default Consumer andThen(Consumer after)
}

void ArrayList.forEach(Consumer<? super E> action)
```

- `IntConsumer { void accept(int v) }`
- `LongConsumer { void accept(long v) }`
- `DoubleConsumer { void accept(double v) }`

Predicate

```
Predicate<T> {
  boolean test(T t)
  default Predicate and(Predicate other)
  default Predicate or(Predicate other)
  default Predicate negate()
}

boolean ArrayList.removeIf(Predicate<? super E> filter)
```

- `IntPredicate { boolean test(int v) }`
- `LongPredicate { boolean test(long v) }`
- `DoublePredicate { boolean test(double v) }`

Function

```
Function<T,R> {
  R apply(T t)
  default Function andThen(Function after)
  default Function compose(Function before)
  default Function identity()
}

<R> R String.transform(Function<? super String,? extends R> f)
```

- `IntFunction<R> { R apply(int v) }`
- `DoubleFunction<R> { R apply(double v) }`
- `ToLongFunction<T> { long applyAsLong(T t) }`
- `IntToLongFunction { long applyAsLong(int v)`
- ...

UnaryOperator

```
UnaryOperator<T> extends Function<T,T> {
  T apply(T t)
  default UnaryOperator andThen(UnaryOperator after)
  default UnaryOperator compose(UnaryOperator before)
  default UnaryOperator identity()
}
```

- IntUnaryOperator { int apply(int v) }
- LongUnaryOperator { long apply(long v) }
- DoubleUnaryOperator { double apply(double v) }

BiConsumer

```
BiConsumer<T, U> {
  void accept(T t, U u)
  default BiConsumer andThen(BiConsumer after)
}
```

- `ObjIntConsumer<T> { void accept(T t, int v) }`
- `ObjLongConsumer<T> { void accept(T t, long v) }`
- `ObjDoubleConsumer<T> { void accept(T t, double v) }`

BiPredicate

```
BiPredicate<T, U> {
  boolean test(T t, U u)
  default BiPredicate and(BiPredicate other)
  default BiPredicate or(BiPredicate other)
  default BiPredicate negate()
}
```

BiFunction

```
BiFunction<T,U,R> {
  R apply(T t, U u)
  default BiFunction andThen(BiFunction after)
}

ToIntBiFunction<T,U> { int applyAsInt(T t, U u) }
ToLongBiFunction<T,U> { long applyAsLong(T t, U u) }
ToDoubleBiFunction<T,U> { double applyAsDouble(T t, U u) }
```

BinaryOperator

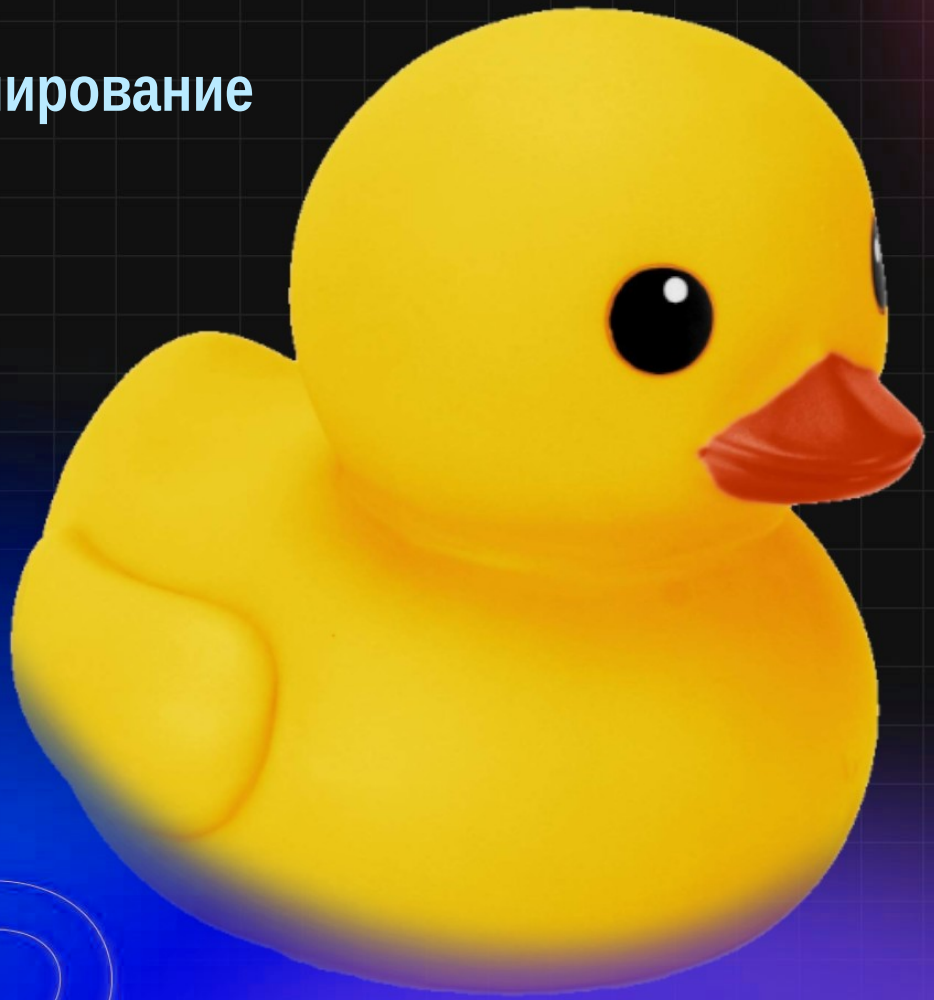
```
BinaryOperator<T> extends BiFunction<T,T> {
  T apply(T t, T t)
  default BinaryOperator andThen(BinaryOperator after)
  static BinaryOperator maxBy(Comparator comp)
  static BinaryOperator minBy(Comparator comp)
}

IntBinaryOperator { int applyAsInt(int v1, int v2) }
LongBinaryOperator { long applyAsLong(long v1, long v2) }
DoubleBinaryOperator { double applyAsDouble(double v1, double v2) }
```

Программирование

2 семестр

2026



Stream API

Пакет java.util.stream

- Конвейерная обработка данных
- Поток — последовательность элементов
- Поток может быть последовательным или параллельным
- Конвейер — последовательность операций

Конвейеры и коллекции

ИТМО

- Отличия конвейера от коллекции
  - Элементы не хранятся
  - Неявная итерация
  - Функциональный стиль — операции не меняют источник
  - Большинство операций работают с λ-выражениями
  - Ленивое выполнение
  - Возможность неограниченного числа элементов

Java

Состав конвейера

- Конвейер =
  - Источник
  - Промежуточные операции (0 или больше)
  - Завершающая операция (одна)

Пример

ИТМО

```
List<String> words
```

```
long count = 0;
for (String s : words) {
  if (s.length() > 5) {
    count++;
  }
}
```

```
long count =
  words.stream()
       .filter(s -> s.length() > 5)
       .count();
```

Java

Источники конвейера

- `Collection.stream()`
- `Arrays.stream(T[] array), .stream(int[] array)`
- `Stream.of(T values)`
- `Stream.Builder.add(T element).build()`
- `IntStream.range(int, int)`
- `Files.lines(Path), BufferedReader.lines()`
- `Random.ints()`
- `Stream.empty()`
- `Stream.generate(Supplier<T> s)`
- `Stream.iterate(T seed, UnaryOperator<T> f)`

Промежуточные операции

ИТМО

- Возвращают поток
- Выполняются "лениво"
  - Конвейер не запускается, пока нет завершающей операции
- Делятся на:
  - Не хранящие состояние (stateless)
    - выполняются вне зависимости от других элементов
  - Хранящие состояние (stateful)
    - выполнение зависит от других элементов (сортировка)

Java

Завершающие операции

- Запускают конвейер
- Возвращают результат
- Либо имеют побочное действие
- Поток прекращает существование

Классы и интерфейсы

ИТМО

- интерфейс BaseStream
  - void close()
  - S parallel()
  - S sequential()
  - S unordered()
  - Iterator iterator()
  - Spliterator spliterator()
- Интерфейс Stream<T>
- Интерфейсы IntStream, LongStream, DoubleStream

Java

Spliterator

- Параллельный Iterator
  - Spliterator trySplit() — возвращает часть элементов как отдельный сплитератор
  - boolean tryAdvance(Consumer action) — выполнить операцию для очередного элемента
  - void forEachRemaining(Consumer action) — выполнить операцию для всех оставшихся элементов

Промежуточные операции (stateless)

ИТМО

- **Stream<T> filter (Predicate<T> p)**
  - возвращает поток из элементов, соответствующих условию
- **Stream<R> map(Function<T,R> mapper)**
  - преобразует поток элементов T в поток элементов R
- Stream<R> flatMap(Function <T, Stream<R>> mapper)
  - преобразует каждый элемент потока T в поток элементов R
- Stream<T> peek(Consumer<T> action)
  - выполняет действие для каждого элемента потока T

Java

Промежуточные операции (stateful)

ИТМО

- Stream<T> distinct()
  - возвращает поток неповторяющихся элементов
- Stream<T> sorted(Comparator<T> comp)
  - возвращает отсортированный поток
- Stream<T> limit(long size)
  - возвращает усеченный поток из size элементов
- Stream<T> skip(long n)
  - возвращает поток, пропустив n элементов

Java

Завершающие операции

- void forEach(Consumer<T> action)
- void forEachOrdered(Consumer<T> action)
  - выполняет действие для каждого элемента потока
  - второй вариант гарантирует сохранение порядка элементов
- Optional<T> min(), Optional<T> max()
  - возвращают минимальный и максимальный элементы,
- long count(), int (long, double) sum()
  - возвращают количество и сумму элементо
- OptionalInt, OptionalLong, OptionalDouble
  - int getAsInt(), long getAsLong(), double getAsDouble()

Класс java.util.Optional<T>

ИТМО

- Оболочка: содержит или не содержит значение
- boolean isPresent() - true, если значение есть
- T get() - возвращает значение
- Optional<T> of(T value) — возвращает оболочку со значением
- T orElse(T other) — возвращает значение или other
- void ifPresent(Consumer<T> action) - выполняет действие, если есть значение

Java

Завершающие операции

- `T reduce(T identity, BinaryOperator<T> accumulator)`

```
.stream
.reduce((a, b) -> a * b)  // подсчет произведения
```

- `R collect(supplier, accumulator, combiner)`
- `R collect(Collector<? super T,A,R> collector)`

```
.stream
.collect(Collectors.toList());
```

- `Object[] toArray()`
- `List toList()`

Завершающие операции

- интерфейс Collector<T,A,R>
  - T - входные элементы, R - результат, A - аккумулятор
- класс Collectors
  - **toCollection**(Supplier factory), **toList**(), **toSet**()
  - **toMap**(Function k, Function v, *BinaryOperator merge, Supplier factory*)
  - **joining**(*String delimiter, String prefix, String suffix*)
  - **mapping**(Function<T,U> mapper, Collector<U> s)
  - **minBy**(Comparator), **maxBy**(Comparator)
  - **counting**(), **summingDouble(), averagingDouble()**
  - **reducing**(*identity, Function<T,U> mapper*, BinaryOperator op)
  - **groupingBy**(Function<T,K> classifier)
  - **partitioningBy**(Predicate<T> predicate)

Проверки

- boolean anyMatch(Predicate<T> p)
  - истина, если условие выполняется хотя бы для одного элемента
  - При нахождении первого совпадения прекращает проверку
- boolean allMatch(Predicate<T> p)
  - истина, если условие выполняется для всех элементов.
  - При нахождении первого несовпадения прекращает проверку
- boolean noneMatch(Predicate<T> p)
  - истина, если условие не выполняется ни для одного элемента.
  - При нахождении первого совпадения прекращает проверку

Ленивое выполнение

ИТМО

```
var list = new ArrayList<>(List.of(1,2,3));
var stream = list
    .stream()
    .peek(System.out::println)
    .filter(i -> i % 2 == 0)
    .peek(i -> System.out.println("> "+i));

System.out.println("Ready");
list.add(4);
long count = stream.count();
System.out.println("Count: " + count);
```

Java

Ленивое выполнение

ИТМО

```
var list = new ArrayList<>(List.of(1,2,3));
var stream = list
    .stream()
    .peek(System.out::println)
    .filter(i -> i % 2 == 0)
    .peek(i -> System.out.println("> "+i));

System.out.println("Ready");
list.add(4);
long count = stream.count();
System.out.println("Count: " + count);
```

Java

Ленивое выполнение

ИТМО

```
var list = new ArrayList<>(List.of(1,2,3));
var stream = list
    .stream()
    .peek(System.out::println)
    .filter(i -> i % 2 == 0)
    .peek(i -> System.out.println("> "+i));

System.out.println("Ready");
list.add(4);
long count = stream.count();
System.out.println("Count: " + count);
```

```
Ready
1
2
> 2
3
4
> 4
Count: 2
```

Java

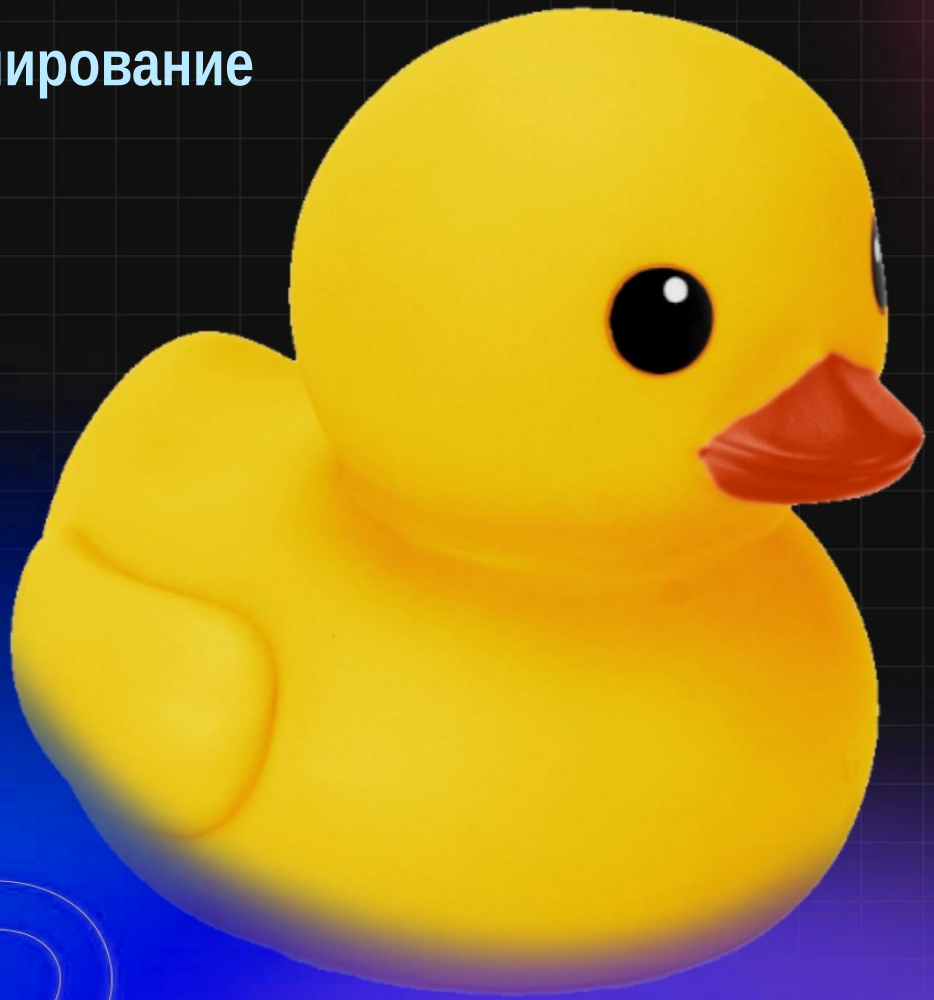
Пример

```
public static void main(String[] args) {
  List sortedArgs =
    Arrays.stream(args)
          .filter(s -> s.length() < 5)
          .map(String::toUpperCase)
          .sorted()
          .collect(Collectors.toList());
}
```

Программирование

2 семестр

2026



Взаимодействие с базами данных

Базы даннных

- БД — структурно организованные данные о предметной области, хранящиеся вместе с информацией о данных и их взаимосвязях.
- СУБД — вычислительная система для создания и использования баз данных.
- Реляционная БД — база данных, основанная на реляционной модели данных.

Реляционная модель

- Отношение (relation) — структура данных, состоящая из заголовка и тела.
  - Заголовок отношения — множество атрибутов
  - Тело отношения — множество кортежей, содержащих значения атрибутов
- Свойства отношения
  - Каждый атрибут имеет тип, значения соответствуют типу
  - Атрибуты не повторяются, и их порядок не имеет значения
  - Кортежи не повторяются, и их порядок не имеет значения

Отношение и таблица

Отношение
Таблица

Атрибуты
Столбцы

Заголовок

Кортежи
Строки

A	B	C	D	E
vx	wx	1	0.0	xyz
vy	wy	2	0.1	yzx
vz	wz	3	0.2	zxy

Тело

Значения

Пример базы данных

students

student_id	name	group
289001	Иванов Петр	P3110
289002	Петров Сидор	P3130
289999	Сидоров Иван	R3140

groups

group	faculty
P3110	ПИиКТ
P3130	ПИиКТ
R3140	СУиР

grades

student_id	course_id	grade
289001	1	A
289001	2	B
289002	1	E
289002	2	A
289999	1	C
288999	3	D

courses

course_id	name	semester	type
1	Программирование	1	З
2	ОПД	2	Э
3	Физика	2	Э

Язык запросов

- SQL (Structured Query Language) — декларативный язык для описания, изменения и получения данных из реляционных баз данных.

CREATE TABLE

- DDL — Data definition language

```
CREATE TABLE weather (
    city      VARCHAR(80),
    temp_lo   INT,        -- low temperature
    temp_hi   INT,        -- high temperature
    prcp      REAL,       -- precipitation
    date      DATE
);

CREATE TABLE IF NOT EXISTS persons

DROP TABLE weather;
```

Ограничения

- Ограничения
  - Типы данных
    - INT, SMALLINT, REAL, DOUBLE, CHAR(n), VARCHAR(n), DATE, TIME, ...
  - Возможные значения в столбце
    - NOT NULL, UNIQUE, CHECK (age >= 18)
  - Ключи
    - Первичный ключ
      - PRIMARY KEY (UNIQUE, NOT NULL)
    - Внешние ключи
      - FOREIGN KEY (REFERENCES)

DML

- DML — Data manipulation language
  - `INSERT`
  - `UPDATE`
  - `DELETE`

INSERT

- Вставка данных в таблицу

```
INSERT INTO weather VALUES ('Oslo', 46, 50, 0.25, '2021-11-27');

INSERT INTO weather (date, city, temp_hi, temp_lo)
    VALUES ('2021-11-29', 'Helsinki', 54, 37);

COPY persons TO file;

COPY persons FROM file;
```

UPDATE

- Обновление данных в таблице

```
UPDATE weather
    SET temp_hi = temp_hi - 2,  temp_lo = temp_lo - 2
    WHERE date > '2021-11-28';
```

DELETE

- Удаление данных из таблицы

```
DELETE FROM weather WHERE city = 'Oslo';

DELETE FROM persons;
```

SELECT

- Выборка данных - запрос

```
SELECT * FROM students;
SELECT name, group FROM students;
SELECT * FROM students WHERE group = 'P3110';
SELECT * FROM students ORDER BY name;
SELECT DISTINCT type FROM courses;
SELECT COUNT(*) FROM students;
```

JOIN

- Соединение таблиц

```
SELECT name, faculty FROM students JOIN groups
    ON students.group = groups.group;  --  USING(group);
```

students

student_id	name	group
289001	Иванов Петр	P3110
289002	Петров Сидор	P3130
289999	Сидоров Иван	R3140

groups

group	faculty
P3110	ПИиКТ
P3130	ПИиКТ
R3140	СУиР

JOIN

student_id	name	group	faculty
289001	Иванов Петр	P3110	ПИиКТ
289002	Петров Сидор	P3130	ПИиКТ
289999	Сидоров Иван	R3140	СУиР

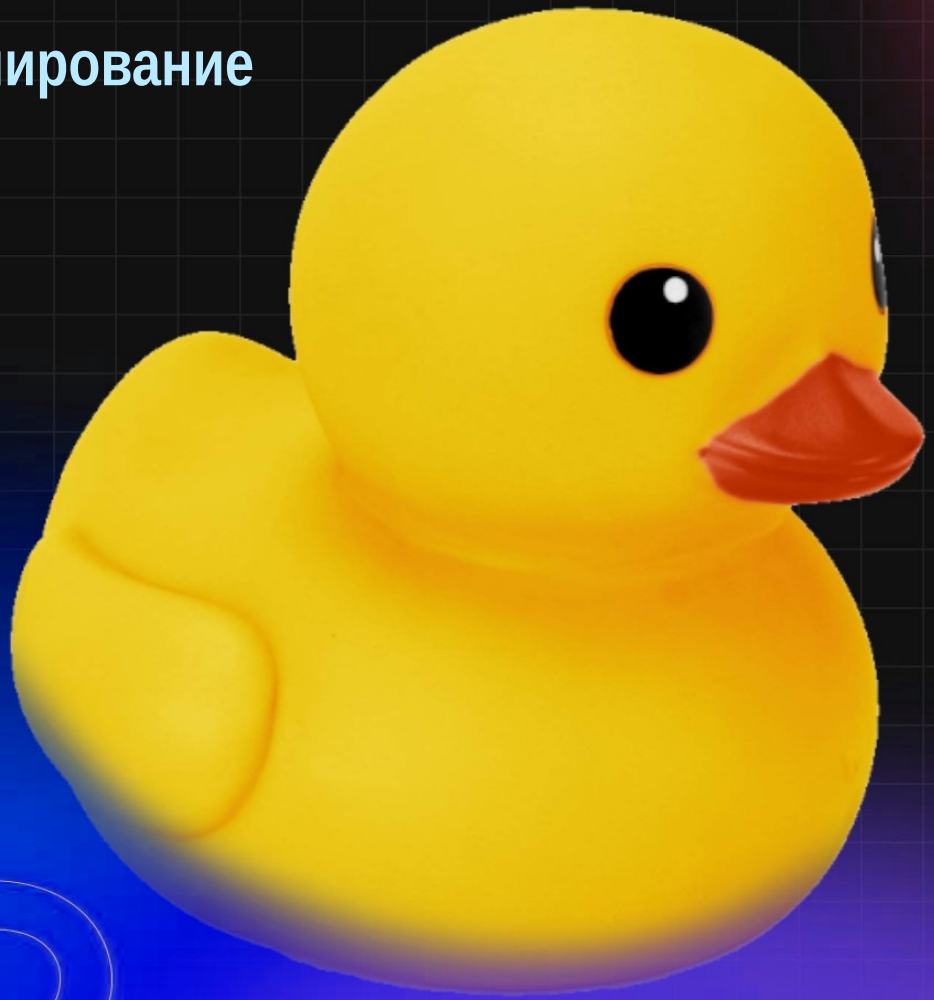
СУБД PostgreSQL

- helios
  - `psql -h pg studs`
  - пароль в файле `.pgpass`
    - `host:port:dbname:login:PaSsw0rD`
  - `драйвер - /usr/local/share/java/classes/postgresql.jar`
- https://postgresql.org
  - \? - помощь по командам psql
  - \h - помощь по командам SQL

Программирование

2 семестр

2026



Основы JDBC

Работа с БД

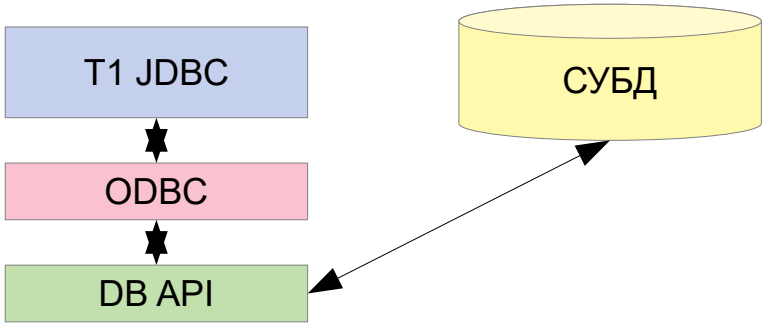
- Много разных СУБД
  - API для каждой базы отдельно
  - единый интерфейс работы с базами + драйвер для конкретной базы
- Реализации
  - ODBC — Open Database Connectivity
  - JDBC — Java Database Connectivity

JDBC

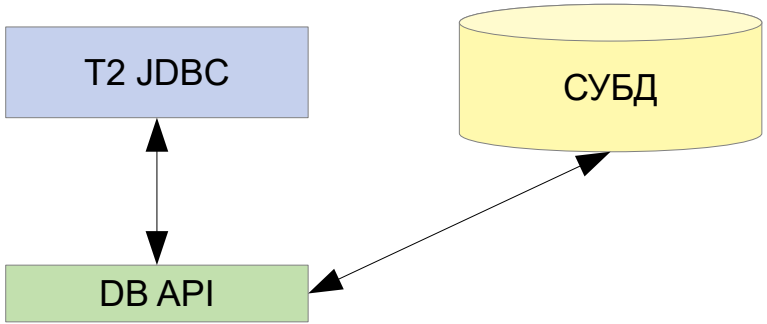
- JDBC — Java DataBase Connectivity
- JDBC API — высокоуровневый интерфейс для доступа к данным
- JDBC Driver API — низкоуровневый интерфейс для создания драйверов
- Пакеты java.sql (Core) и javax.sql (Extension)
- Стандарт взаимодействия с СУБД

Типы взаимодействия

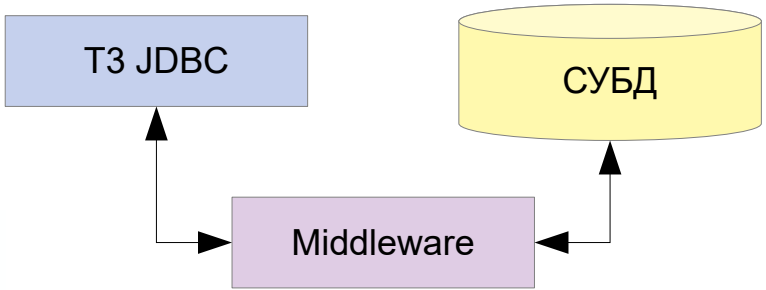
- Тип 1 — мост ODBC



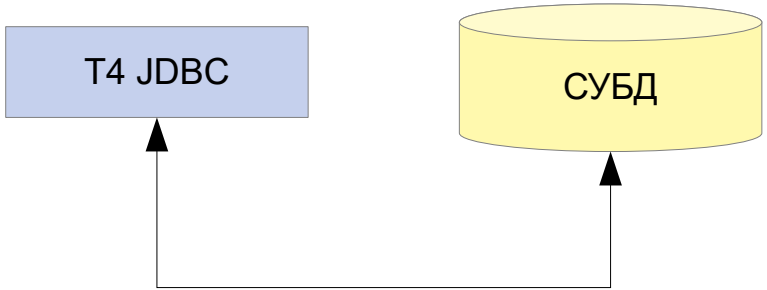
- Тип 2 — DB API



- Тип 3 — middleware



- Тип 4 — pure Java



Процесс взаимодействия

ИТМО

```
Connection connection = DriverManager.getConnection(…);

  Statement statement = connection.createStatement();

    ResultSet resultSet = statement.executeQuery("SELECT …");

    while (resultSet.next()) {
    // получение и обработка данных
    }

    resultSet.close();
  statement.close();
connection.close();
```

Java

Интерфейс Driver

- java.sql.Driver
  - Отвечает за связь с БД
  - Метод Connection connect(String url, Properties info)
  - Используется для написания драйверов для СУБД

Класс DriverManager

- Управляет списком драйверов
- Загрузка драйвера
  - Class.forName()
  - jdbc.drivers=
- Неявная загрузка с помощью ServiceLoader
  - META-INF/services/java.sql.Driver

Класс DriverManager

- Метод Connection getConnection(String url, ...)
  - url = jdbc:*protocol*://*host*:*port*/*database*
    - jdbc:postgresql://db:5432/studs
  - getConnection(String url, Properties info)
    - Properties info = new Properties();
    - info.load(new FileInputStream("db.cfg");

      файл db.cfg

```
user = s999999
password = sss999
```

  - getConnection(Stiring url, String name, String pass)

Интерфейс Connection

- Абстракция соединения (сессия)
  - методы:
    - Statement createStatement()
    - PreparedStatement prepareStatement(String sql)
    - CallableStatement prepareCall(String sql)

    - DatabaseMetaData getMetaData()

Семейство интерфейсов Statement

- Statement
  - Статический SQL-запрос
  - Запрос передается как параметр при выполнении

```
String query = "SELECT * FROM table WHERE id = 15";
Statement st = connection.createStatement();
st.executeQuery(query);
```

Семейство интерфейсов Statement

- **PreparedStatement** extends Statement
  - **Динамический запрос** с подстановкой
  - Запрос передается как параметр при создании

```
String query = "SELECT * FROM table WHERE id = ?";
PreparedStatement ps = connection.prepareStatement(query);
ps.setInt(1, 15); // 1 — номер параметра, 15 — значение
SELECT * FROM table WHERE id = 15
```

- Предотвращает SQL-инъекции

Семейство интерфейсов Statement

- CallableStatement extends PreparedStatement
  - Вызов хранимой процедуры

    SQL: CREATE PROCEDURE

```
cs = prepareCall("CALL getResult (?)");
cs.setInt(1, 15);
cs.registerOutParameter(1, Types.INTEGER);
...
int result = cs.getInt(1);
CALL getResult(15);
```

Выполнение запросов - executeQuery

- ResultSet executeQuery(String sql) // Statement
- ResultSet executeQuery() // PreparedStatement (Callable)
  - для выполнения SELECT
  - возвращает ResultSet

```
ResultSet resultSet = statement.executeQuery();
```

Выполнение запросов - executeUpdate

- int executeUpdate(String sql) // Statement
- int executeUpdate() // PreparedStatement (Callable)
  - для выполнения INSERT, UPDATE, DELETE + DDL
  - возвращает количество измененных строк
  - Для запросов DDL возвращает 0

```
int updateCount = statement.executeQuery();
```

Выполнение запросов - execute

- boolean execute(String sql) // Statement
- boolean execute() // PreparedStatement (Callable)
  - для выполнения любых запросов

```
if (statement.execute()) {
    ResultSet resultSet = statement.getResultSet();
} else {
    int updateCount = statement.getUpdateCount();
}
```

Транзакции

- Connection
  - setAutoCommit(true/false)
  - commit()
  - rollback()
  - setSavepoint()

Транзакции

- Connection
  - setAutoCommit(true/false)
  - commit()
  - rollback()
  - setSavepoint()
- Statement
  - addBatch(String sql)
  - clearBatch()
  - executeBatch()

Интерфейс ResultSet

- Получение данных из ResultSet

```
ResultSet rs = preparedStatement.executeQuery();

while (rs.next()) {
    String name = rs.getString(1); // по номеру столбца
    int id = rs.getInt("id");      // по имени столбца
}
```

Настройка типа ResultSet

- ResultSet
  - Connection.**createStatement**(sql, **type, concurrency, holdability**)
  - ResultSetType
    - **TYPE_FORWARD_ONLY**
    - TYPE_SCROLL_INSENSITIVE
    - TYPE_SCROLL_SENSITIVE
  - ResultSetConcurrency,
    - **CONCUR_READ_ONLY**
    - CONCUR_UPDATABLE
  - ResultSetHoldability
    - HOLD_CURSORS_OVER_COMMIT
    - CLOSE_CURSORS_AT_COMMIT

Интерфейс ResultSet

- Навигация
  - `next()`
  - `previous()`
  - `first()`
  - `last()`
  - `beforeFirst()`
  - `afterLast()`
  - `relative(int row)`
  - `absolute(int row)`
  - `moveToInsertRow()`

Интерфейс ResultSet

- Получение данных
  - getString(int)
  - getString(String)
  - getInt(int)
  - getInt(String)
  - getBoolean
  - getLong
  - getDouble
  - getArray (SQL Array)
  - getDate
  - getTimestamp
  - getReader
  - ...

Интерфейс ResultSet

- Обновление строк
  - `updateInt(String, int)`
  - `updateInt(int, int)`
  - `updateString(String, String)`
  - `updateString(int, String)`
  - `updateRow()`
- Добавление строк
  - `moveToInsertRow()`
  - `updateInt(String, int)`
  - `insertRow()`

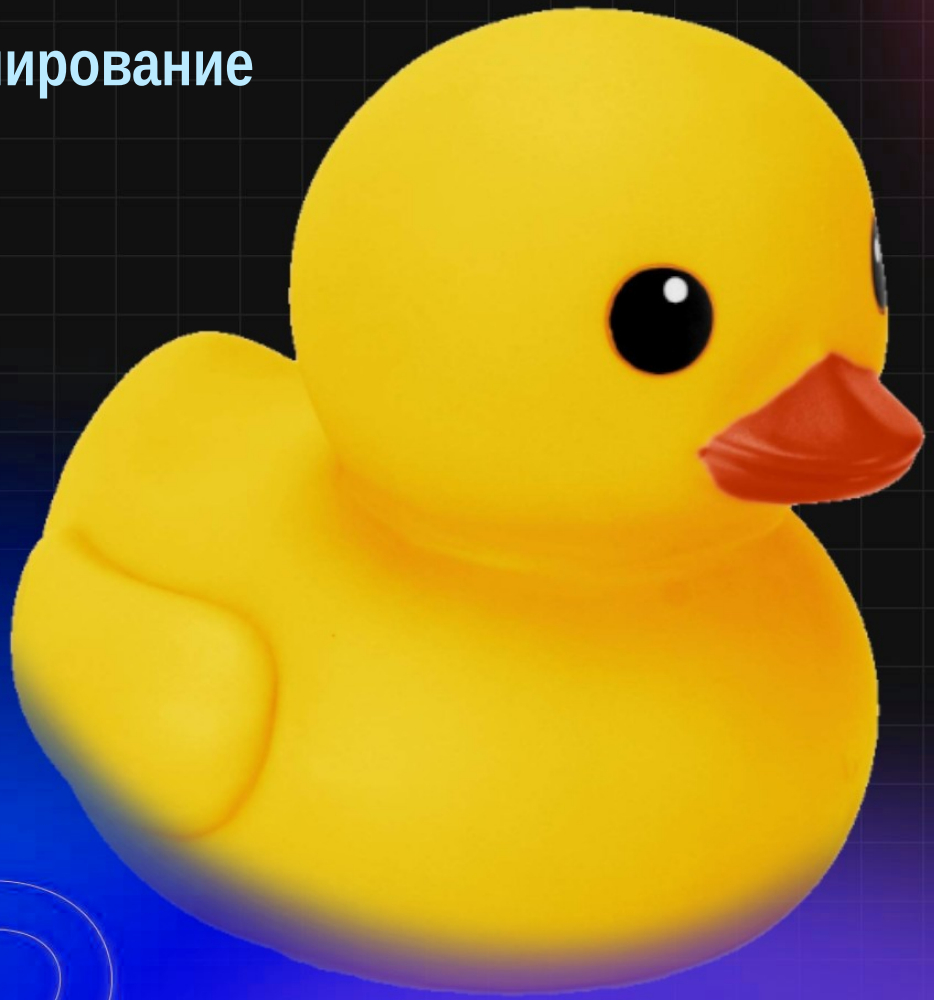
Метаданные

- ResultSetMetaData ResultSet.getMetaData()
  - `getTableName()`
  - `getColumnCount()`
  - `getColumnName(int n)`
  - `getColumnType(int n)`
- DatabaseMetaData Connection.getMetaData()
  - `getCatalogs()`
  - `getTables()`
  - `getSchemas()`

Программирование

2 семестр

2026



Расширения JDBC

Интерфейс DataSource

- javax.sql.DataSource
  - Позволяет получить соединение с БД
  - org.postgresql.ds.PGSimpleDataSource
- ConnectionPoolDataSource
  - Поддержка пула соединений
  - org.postgresql.ds.PGPoolingDataSource
- XADataSource
  - Поддержка распределенных транзакций

Простой источник данных

```
import org.postgresql.ds.PGSimpleDataSource;

PGSimpleDataSource ds = new PGSimpleDataSource();

ds.setServerName(...);
ds.setDatabaseName(...);
ds.setUser(...);
ds.setPassword(...);

Connection conn = ds.getConnection();
```

Поддержка JNDI

```
import javax.naming.*;

Context context = new InitialContext();
DataSource ds = ...          // подготовка DataSource
context.bind("testDB", ds); // привязка к контексту

// получение DataSource из контекста
Context context = new InitialContext();
DataSource ds = (DataSource)context.lookup("testDB");
```

Интерфейс RowSet

- javax.rowset.*
- Единый интерфейс для всех операций

- RowSet extends ResultSet
  - setUrl(),
  - setUsername(),
  - setPassword(),
  - setCommand("Select * from ...");
  - execute()
  - next()
  - getXXX()

```
var factory = RowSetProvider.newFactory()
var rowset = factory.createJdbcRowSet();
```

Фабрики RowSet

- RowSetFactory factory = RowSetProvider.newFactory();
  - factory.createJdbcRowSet();
  - factory.createCachedRowSet();
  - factory.createWebRowSet();
  - factory.createFilteredRowSet();
  - factory.createJoinRowSet();

Разновидности RowSet. JdbcRowSet

- JdbcRowSet — простая разновидность RowSet
  - Поддерживает соединение с базой данных
  - По умолчанию:
    - Тип: SCROLL_INSENSITIVE
    - Конкурентность: CONCUR_UPDATABLE
    - Включено экранирование спецсимволов

Разновидности RowSet. JdbcRowSet

- JdbcRowSet — простая разновидность RowSet

```
JdbcRowSet rs = factory.createJdbcRowSet();
rs.setUrl(""); …
rs.setCommand("Select * from users");
rs.execute();
rs.last();
rs.getInt("id");
rs.updateString("name", "Pupkin");
rs.UpdateRow();
```

Разновидности RowSet. CachedRowSet

- CachedRowSet
  - Результат запроса может кэшироваться
  - Синхронизация с базой
  - Разрешение конфликтов
  - acceptChanges()

Разновидности RowSet. CachedRowSet

- CachedRowSet

```
CachedRowSet rs = factory.createCachedRowSet();
rs.setUrl(""); …
rs.setCommand("Select * from users");
rs.execute();
rs.last();
rs.getInt("id");
rs.updateString("name", "Pupkin");
rs.acceptChanges();
```

Разновидности RowSet. WebRowSet

- WebRowSet
  - Может записывать и читать результат в виде XML
  - writeXML()
  - readXML()

Разновидности RowSet. WebRowSet

- WebRowSet

```
WebRowSet rs = factory.createWebRowSet();
rs.setUrl(""); …
rs.setCommand("Select * from users");
rs.execute();
rs.writeXML(new FileWriter("data.xml"));
```

Разновидности RowSet. FilteredRowSet

- FilteredRowSet
  - Фильтрация строк (аналог WHERE)
  - setFilter(Predicate p)
- Predicate
  - boolean evaluate(Object value, int/String column)
  - boolean evaluate(RowSet rowset)

Разновидности RowSet. FilteredRowSet

- FilteredRowSet

```
FilteredRowSet rs = factory.createFilteredRowSet();
rs.setUrl(""); …
rs.setCommand("Select * from users");
rs.execute();
Predicate filter = new Predicate();
rs.setFilter(filter);
```

Разновидности RowSet. JoinRowSet

- JoinRowSet
  - Соединение нескольких результатов (JOIN)
  - addRowSet()
  - setJoinType()
  - toCachedRowSet()

Разновидности RowSet. JoinRowSet

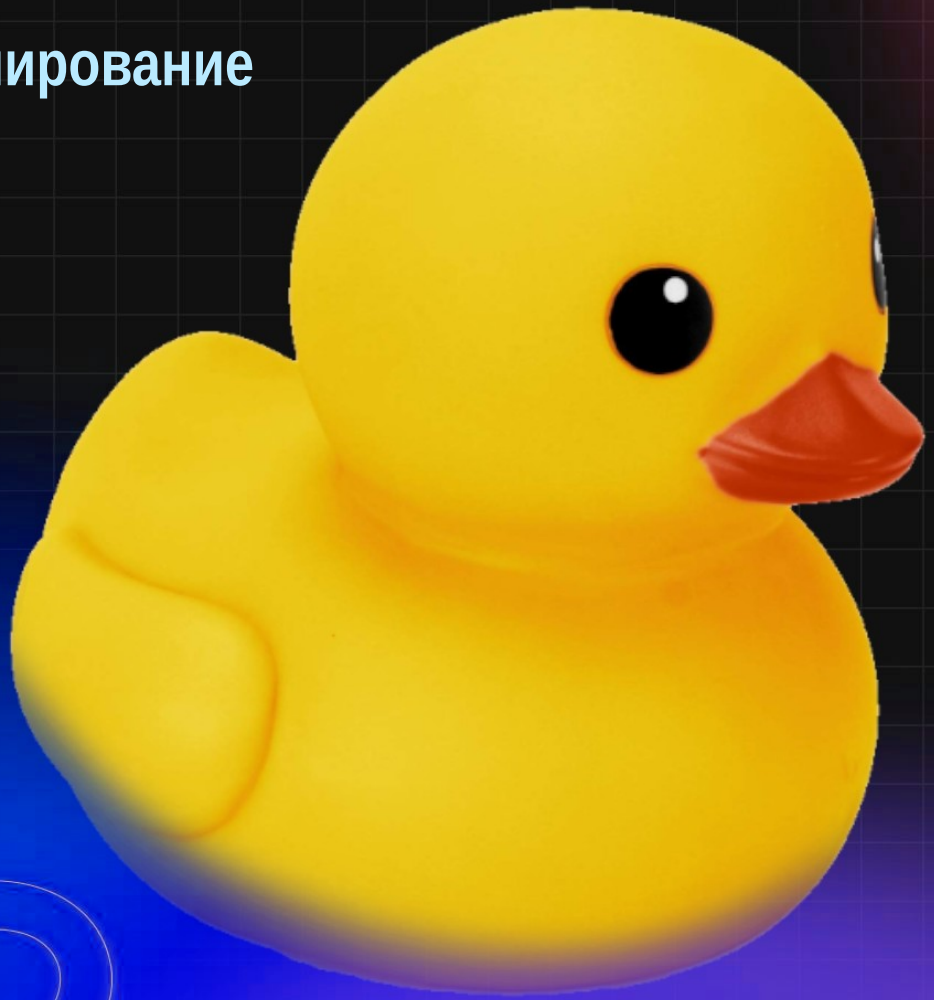
- JoinRowSet

```
JoinRowSet js = factory.createJoinRowSet();
CachedRowSet users; // "Select * from users"
CachedRowSet groups; // "Select * from groups"
users.setMatchColumn("uid");
groups.setMatchColumn("uid");
js.addRowSet(users);
js.addRowSet(groups);
```

Программирование

2 семестр

2026



Провайдеры служб

Провайдеры служб

- Служба или сервис
- Разные реализации
- Нужен механизм поиска и загрузки служб

Шпаргалка на экзамен

```
class CheatSheet {
  public String getAnswer(String question) {
    Map source =
    String answer = source.get(question);
    if (answer != null) return answer;
    else return "Epic fail";
  }
}
```

Нужен источник знаний

```
class CheatSheet {
  public String getAnswer(String question) {
    Map source = knowledge.getSource();
    String answer = source.get(question);
    if (answer != null) return answer;
    else return "Epic fail";
  }
}
interface Knowledge {
  Map<String,String> getSource();
}
```

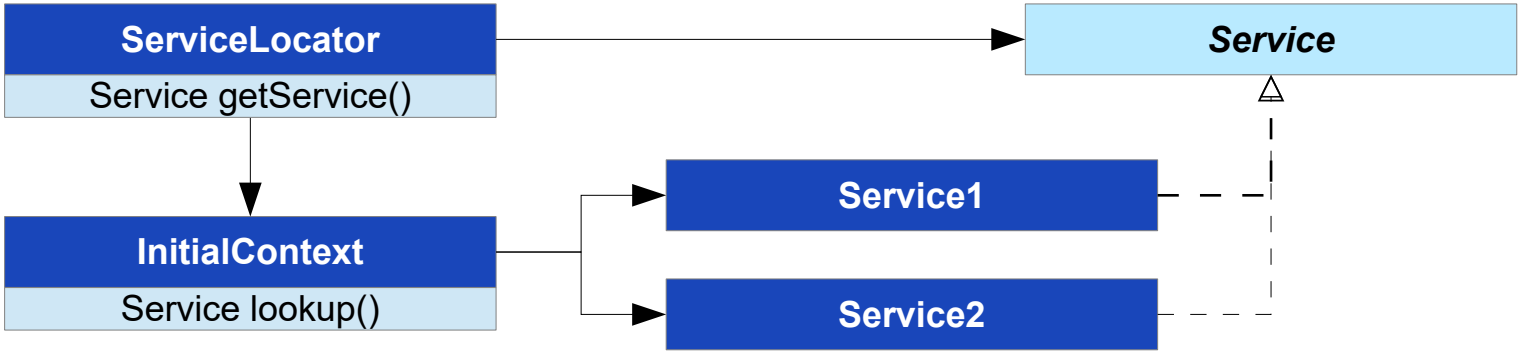
Различные источники

```
class CheatSheet {
  public String getAnswer(String question) {
    Map source = knowledge.getSource();
    String answer = source.get(question);
    if (answer != null) return answer;
    else return "Epic fail";
  }
}
interface Knowledge {
  Map<String,String> getSource();
}
```

```
class Memory implements Knowledge
class Lecture implements Knowledge
class CallFriend implements Knowledge
class Miracle implements Knowledge
```

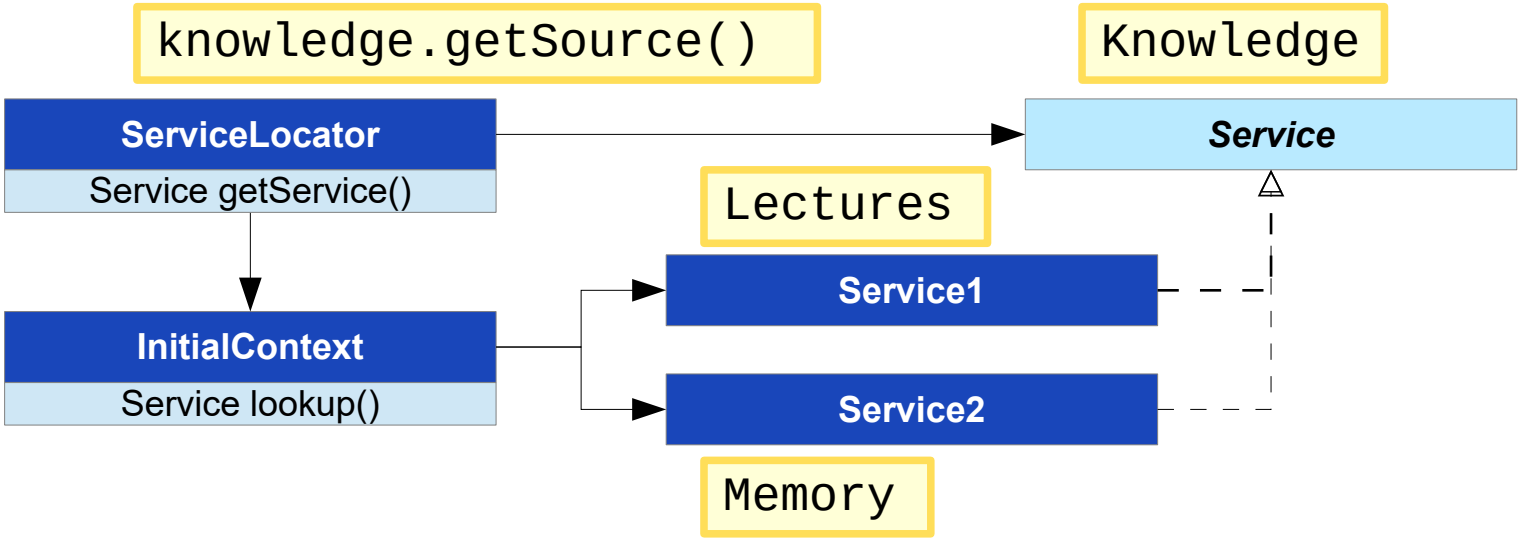
Как передать объект?

- Шаблон ServiceLocator



Как передать объект?

- Шаблон ServiceLocator



Реализация в Java

- Класс java.util.ServiceLoader<Service>
  - static ServiceLoader<Service> load(Service.class)
  - Iterator<Service> iterator()

Реализация в Java

- **interface** spi.Service
  - public void execute();
- **public class** spi.DefaultServiceImpl {
  - **public DefaultServiceImpl() { ... }**
  - public void execute() { ... }
- service.jar
  - **каталог META-INF/services/**
    - файл **spi.Service**
      - **spi.DefaultServiceImpl**

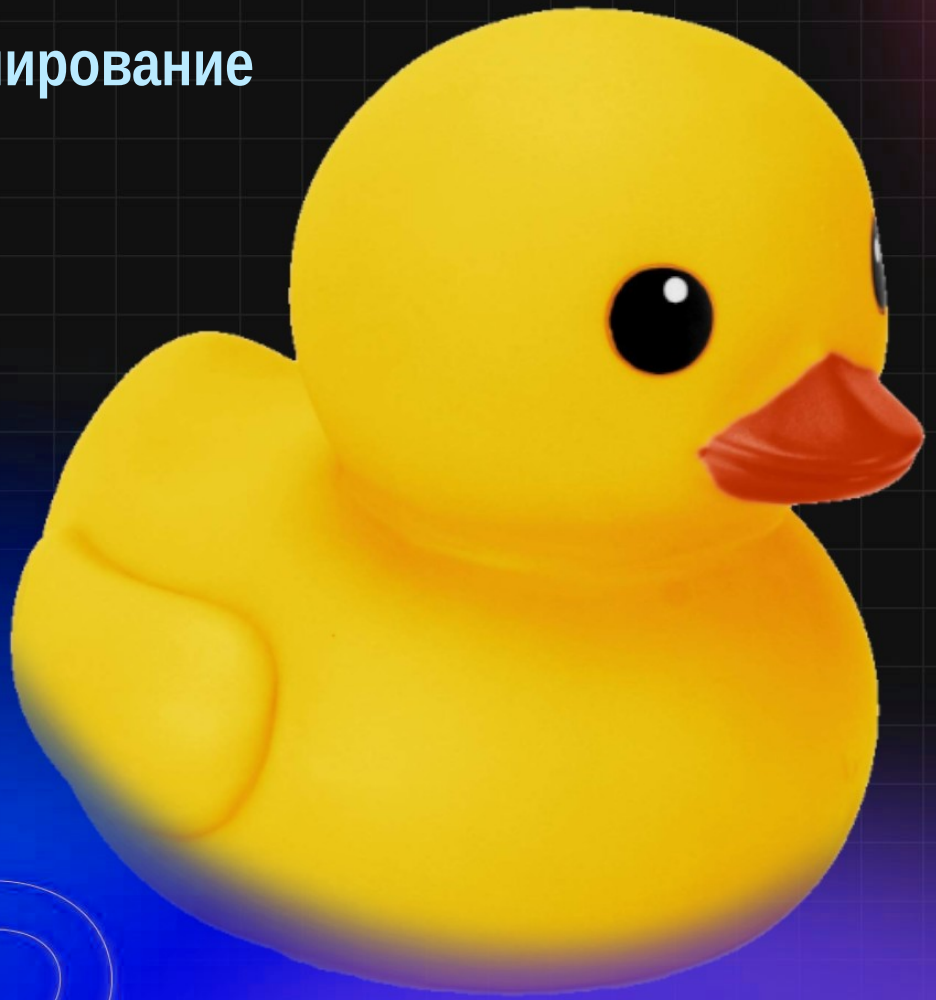
Провайдеры

- java.sql.DriverManager
- java.nio.file.spi.FileSystemProvider
- java.nio.channels.spi.AsynchronousChannelProvider
- java.nio.channels.spi.SelectorProvider
- java.nio.charset.spi.CharsetProvider
- java.text.spi.DateTimeFormatProvider
- java.text.spi.NumberFormatProvider
- java.util.spi.CalendarDataProvider

Программирование

2 семестр

2026



Модули

Java до модулей - классы, пакеты, JARы

- Классы
  - public class - доступен всем
  - объединяются в пакеты (доступ внутри пакета)
  - упаковываются в JAR-архивы
- Пакеты ≈ каталоги
  - пакет и JAR - разные сущности
  - файл package-info.java / .class

Java до модулей - JAR-hell

ИТМО

- classpath
  - просмотр всех путей, загрузка первого найденного класса
- конфликт версий
  - не та версия класса, классы разных версий
- кастомные загрузчики классов
- нет зависимостей на уровне JVM
- JRE - Java Runtime Environment
  - rt.jar - содержит всю стандартную библиотеку

Модули

- Явные зависимости (requires и exports)
- модульная библиотека
  - базовый модуль java.base
  - платформенные модули (все остальные)
  - новый формат артефактов - jmod
- jlink - для сборки кастомного образа
  - rt.jar удален в Java 11
- модульный JAR (+module-info.class)

Модули - компиляция и запуск

ИТМО

- Декларация модуля - module-info.java / .class

```
module my.mod {
    requires java.base;
    requires java.sql;
    exports my.package;
}
```

- Опции javac и java

```
javac --module-path mods:libs -d ./out

java --module-path mods:libs --module my.mod
```

Модули - доступ

- module m0 - наш модуль
- **exports** p1 - экспортирует пакет p1 всем (доступ во время компиляции к открытым сущностям пакета)
- **exports** p1 to m1 - доступ только для модуля m1
- **opens** p2 - открывает пакет p2 всем (доступ с помощью рефлексии во время выполнения ко всем сущностям)
- **opens** p2 **to** m2 - открыт только для модуля m2

Модули - зависимости

- module m0 - наш модуль
- **requires** m1 - m0 требует m1 при компиляции и выполнении
- **requires transitive** m2 - m0 требует m2 и модули, зависящие от m0, тоже требуют m2
- **requires static** m3 - m0 требует m3 при компиляции, но не обязательно при выполнении

Модули - сервисы

- module m0 - наш модуль
- **uses** i1 - использует сервис с интерфейсом i1 (может загружать его с помощью ServiceLoader)
- **provides** i2 with c1, c2 - предоставляет интерфейс сервиса i2, который реализуют конкретные классы c1 и c2

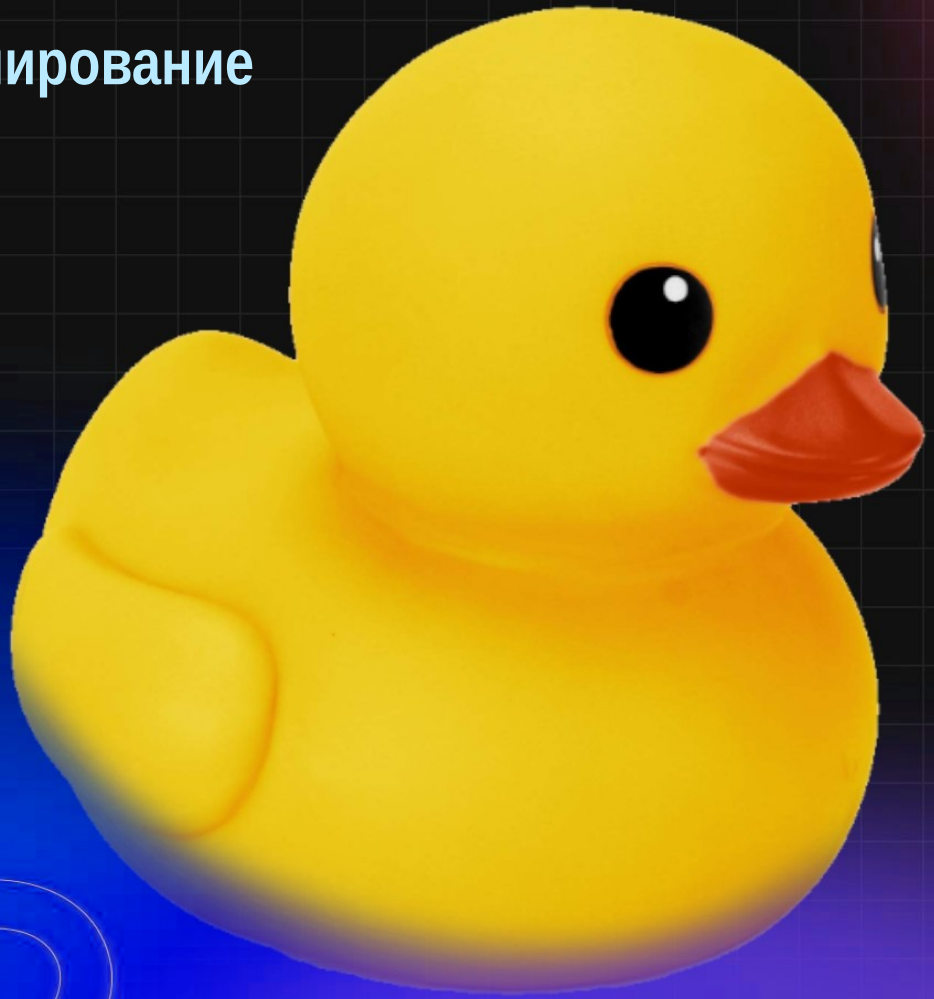
Модули - совместимость

- Безымянный модуль
  - автоэкспорт всех своих пакетов
  - имеет доступ ко всем другим модулям
  - используется classpath

Программирование

2 семестр

2026



Безопасное хранение паролей

Хранение пароля в базе

- CREATE TABLE users (varchar name, varchar password);

Пароль нельзя хранить в открытом виде

- Доступ к базе
- Одинаковые пароли
- Резервное копирование
- SQL-инъекции и другие методы

SQL-инъекции

```
CREATE TABLE users (varchar name, varchar password);
CREATE TABLE cats (varchar cat, varchar owner);
q1 = "SELECT * FROM users WHERE name = '%s' AND password = '%s';";
q2 = "SELECT cat, owner FROM cats WHERE owner = '%s';";
username = Scanner.readLine(); password = Scanner.readLine();
statement.executeQuery(String.format(q1, username, password));
statement.executeQuery(String.format(q2, username));
```

(C) https://simonscat.com

SQL-инъекции

```
CREATE TABLE users (varchar name, varchar password);
CREATE TABLE cats (varchar cat, varchar owner);
q1 = "SELECT * FROM users WHERE name = '%s' AND password = '%s';";
q2 = "SELECT cat, owner FROM cats WHERE owner = '%s';";
```

Name: **Simon** Password: **123456**

```
SELECT * FROM users WHERE name = 'Simon' AND password = '123456';
SELECT cat FROM cats WHERE owner = 'Simon';
```

(C) https://simonscat.com

SQL-инъекции

```
CREATE TABLE users (varchar name, varchar password);
CREATE TABLE cats (varchar cat, varchar owner);
q1 = "SELECT * FROM users WHERE name = '%s' AND password = '%s';";
q2 = "SELECT cat, owner FROM cats WHERE owner = '%s';";
```

Name: `Simon';--` Password: `1`

```
SELECT * FROM users WHERE name = 'Simon';--' AND password = '1';
Login OK!
```

(C) https://simonscat.com

SQL-инъекции

```
CREATE TABLE users (varchar name, varchar password);
CREATE TABLE cats (varchar cat, varchar owner);
q1 = "SELECT * FROM users WHERE name = '%s' AND password = '%s';";
q2 = "SELECT cat, owner FROM cats WHERE owner = '%s';";
```

Name: `' OR ''='` Password: `' OR ''='`

```
SELECT * FROM users WHERE name = '' OR ''='' AND password = '' OR ''=''
Login OK!
```

(C) https://simonscat.com

SQL-инъекции

```
CREATE TABLE users (varchar name, varchar password);
CREATE TABLE cats (varchar cat, varchar owner);
q1 = "SELECT * FROM users WHERE name = '%s' AND password = '%s';";
q2 = "SELECT cat, owner FROM cats WHERE owner = '%s';";
```

Name: `' UNION SELECT name,password FROM users WHERE name<>'`

```
SELECT cat, owner FROM cats WHERE owner = ''
UNION SELECT name,password FROM users WHERE name<>'';
```

(C) https://simonscat.com

SQL-инъекции

ИТМО

```
CREATE TABLE users (varchar name, varchar password);
CREATE TABLE cats (varchar cat, varchar owner);
q1 = "SELECT * FROM users WHERE name = '%s' AND password = '%s';";
q2 = "SELECT cat, owner FROM cats WHERE owner = '%s';";
```

Name: `' UNION SELECT name,password FROM users WHERE name<>'`

```
SELECT cat, owner FROM cats WHERE owner = ''
UNION SELECT name,password FROM users WHERE name<>'';
Simon's Cat     Simon
admin           @#p1nk_m0nkey$%
```

(C) https://simonscat.com

Лучше хранить хеш

- Вместо пароля — хеш пароля
- Функция необратимая
- Минимальное число коллизий
- MD2, MD5, SHA-1, SHA-224, SHA-256, SHA-384, SHA-512
- md5("hello") = 5d41402abc4b2a76b9719d911017c592

Коллизии

- Дни рождения
- В группе > 23 человек
- Вероятность совпадения дня рождения?

Коллизии

- Дни рождения

Лучше хранить хеш

- Вместо пароля — хеш пароля
- Функция необратимая — но можно подобрать
- Минимальное число коллизий — тоже можно подобрать
- MD2, MD5, SHA-1, SHA-224, SHA-256, SHA-384, SHA-512
- md5("hello") = 5d41402abc4b2a76b9719d911017c592
- Словарные атаки

  https://md5.gromweb.com/

  http://reversemd5.com/

Добавим к хешу соль

- Вместо пароля — хеш пароля + соль
- Соль — некая случайная последовательность

md5("hello" + "$Ins50D") = fcac10c41f9f67090e450161db4bca5a

md5("hello" + "HOxc3@") = a893f9063e4314635e05f8c46b547f7a

Добавим к хешу соль

- Вместо пароля — хеш пароля + соль
- Соль — некая случайная последовательность

md5("hello" + "$Ins50D") = fcac10c41f9f67090e450161db4bca5a

md5("hello" + "HOxc3@") = a893f9063e4314635e05f8c46b547f7a

- Храним пароль + соль + алгоритм + сложность
- Все равно — "hello", "12345" — словарные атаки

Добавим перец

- В базе храним хеш пароля + соль
- Перец храним в приложении (отдельно)
- md5("hello" + "kFz<Q%ps" + "$Ins50D")

Пример кода

```
  java.security.MessageDigest
MessageDigest md = MessageDigest.getInstance("MD5");
String user   = Console.readLine()
String passwd = Console.readPassword();
String salt   = getRandomString();
String pepper = "*63&^mVLC(#"
byte[] hash = md.digest(
      (passwd + pepper + salt).getBytes("UTF-8"));

insert into users (user, salt, hash);
```

Исходящие пароли

- Входящие пароли - хеш, соль
- Исходящие пароли - отдельно от кода!
  - Запрашивать при старте
  - Конфигурационный файл с ограничением прав
  - Файлы свойств (Properties)
  - Переменная окружения
  - $HOME/.pgpass